FoodAds AI

Full Project Documentation

AI-Powered Restaurant Marketing Studio

Technical Architecture • Backend • Frontend • AI Services • Deployment

Presented By:

Mohamed Tarek Mohamed

Omar Abdelnasser Elsayed

Marcelino Maged Khalil

Verina Antounious Atya

Momen Mohamed Mahmoud

DEPI Graduation Project

Table of Contents

1. Project Overview

FoodAds AI is an AI-powered restaurant marketing studio. Given a single text prompt, the platform delivers an end-to-end marketing workflow — from concept to publish-ready assets — for any restaurant.

What the Platform Does

- Enhances the user's raw prompt using an LLM (Groq / LLaMA 3.1).
- Generates a photorealistic food image using Stable Diffusion (Base or LoRA-fine-tuned).
- Generates a full multi-channel marketing campaign (Google Ads, Instagram, TikTok, Email, SMS, Push Notification, and a Content Calendar).
- Saves campaign history with embedded image previews.
- Lets users favorite campaigns and download styled HTML reports.

1.1 Business Context & Problem Statement

- **High Marketing Overhead:** Traditional food marketing requires professional photography and copywriting. Hiring agencies or freelancers can cost anywhere from $1,000 to $5,000 monthly, which is prohibitive for small local restaurants.
- **Time Poverty:** Restaurant owners and managers spend over 60+ hours a week handling operations, kitchen staff, and suppliers, leaving virtually zero time to create social media posts or write ad copy.
- **Crucial Visual Appeal:** Food businesses rely entirely on appetite appeal. Generic stock photos alienate customers, while cheap smartphone photos lack the composition and lighting required to convert viewers into paying customers.

1.2 Target Audience

- **Small & Medium Local Restaurants / Cafés:** Family-owned diners, local pizzerias, bistros, and coffee shops looking to handle their own digital presence.
- **Cloud Kitchens & Home Food Businesses:** Businesses operating on delivery-only models that need rapid daily visual and text content to highlight daily specials.
- **Freelance SMMs & Local Agencies:** Social media managers handling multiple food brands who need to scale their campaign drafting and image creation processes from hours to seconds.

1.3 Value Proposition

- **90%+ Cost Reduction:** Replaces expensive creative retainers with an automated on-demand AI workspace.
- **Instant Go-To-Market:** Formulates a complete multi-channel campaign (Instagram Feed, Google Search, Email, SMS, Push Notification, 3-day social calendar) in under 30 seconds.
- **Tailored Food Visuals:** Utilizes a custom-fine-tuned Stable Diffusion LoRA specifically optimized for food textures, colors, and restaurant lighting, avoiding the generic or distorted look of standard base AI models.

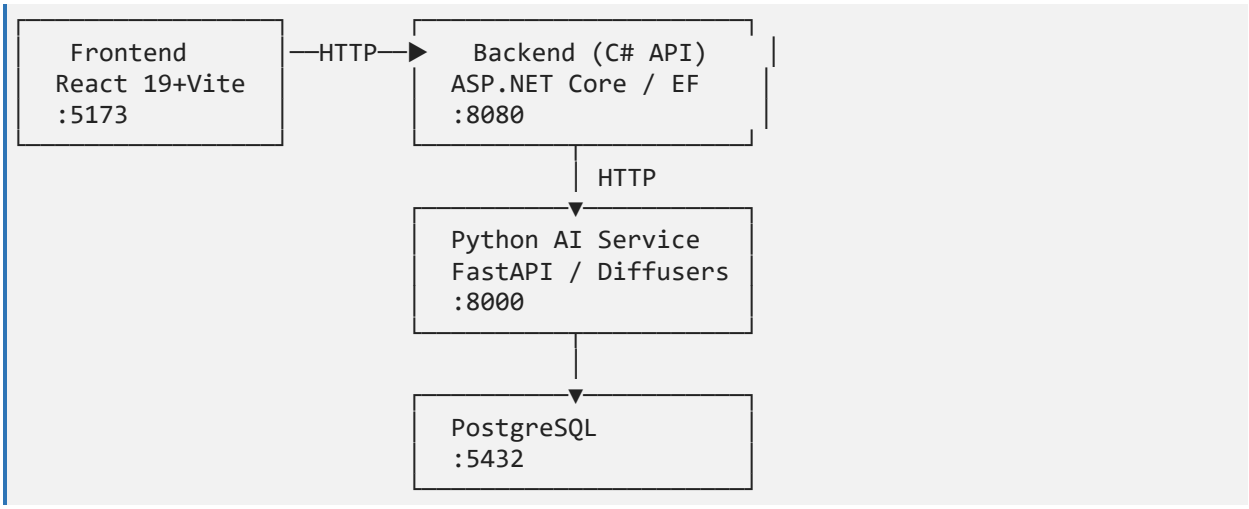
1.4 Market Statistics & ROI Potential

- **Conversion Uplift (DoorDash & Grubhub):** Adding high-quality photos is linked to a 30% increase in sales (Grubhub) and up to 44% higher monthly sales (DoorDash). Including detailed item descriptions alongside photos can boost orders by up to 70%.
- **Customer Behavior (TouchBistro 2025):** Approximately 57% of millennials rely directly on social media platforms when deciding where to dine, and industry reports show up to 90% of all diners research a restaurant online before visiting.
- **Agency Monetization Models:** Freelance social media managers (SMMs) utilizing automated generators to service local food brands charge standard monthly retainers of $300 to $800 per client, allowing a single marketer to scale to $3,000 – $5,000 in Monthly Recurring Revenue (MRR) by automating visual assets and copy production.

2. System Architecture

The system is composed of four services, all containerized with Docker and orchestrated via Docker Compose.

2.1 High-Level Diagram



2.2 Services Summary

Service	Technology	Port
Frontend	React 19, Vite, TypeScript	5173
Backend API	ASP.NET Core 8, EF Core, Npgsql	8080
Python AI Service	FastAPI, Diffusers, Groq SDK	8000
Database	PostgreSQL (latest)	5432

3. Local Development Setup

3.1 Prerequisites

- Node.js 20+
- .NET 8 SDK
- Python 3.11+
- PostgreSQL 15+ running locally (or use Docker for just the DB)
- (Optional) NVIDIA GPU with CUDA 12.1 drivers for hardware-accelerated image generation

3.2 Frontend

```
cd frontend
npm install
npm run dev          # Dev server at http://localhost:5173
npm run build        # Production build to dist/
```

3.3 Backend

```
cd backend
dotnet build
dotnet run --project src/FoodAdsAI.Api
# API runs at http://localhost:8080
# Swagger UI at http://localhost:8080/swagger (Development only)
```

3.4 Python AI Service

```
cd python-ai-service
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
# Health check: GET http://localhost:8000/v1/health
```

3.5 Environment for Local Development

Create a root .env file (or set env vars manually):

```
# Database
ConnectionStrings__FoodAdsAI=Host=localhost;Port=5432;Database=foodadsai;Username=fo
odadsai;Password=foodadsai

# AI Service
AiService__BaseUrl=http://localhost:8000

# JWT
Jwt__Issuer=FoodAdsAI
Jwt__Audience=FoodAdsAI
```

```
Jwt__Secret=your-very-long-secret-key-here
Jwt__AccessTokenMinutes=60
Jwt__RefreshTokenDays=30

# Groq
GROQ_API_KEY=gsk_...

# Stable Diffusion
BASE_MODEL_ID=stablediffusionapi/realistic-vision-v51
LORA_ADAPTER_PATH=D:/FoodAds-Ai
ALLOW_PLACEHOLDER_IMAGES=true      # Use true locally if no GPU
OUTPUT_DIR=D:/FoodAds-Ai/output

# HuggingFace
HF_TOKEN=hf_...
```

3.6 Running Without a GPU

Set `ALLOW_PLACEHOLDER_IMAGES=true` — the AI service will return a styled placeholder image instead of running Stable Diffusion. All LLM features (prompt enhancement, campaign copy) still work normally as long as `GROQ_API_KEY` is set.

---

4. Deployment

4.1 Docker Compose

The full stack is defined in docker-compose.yml at the project root.

Services Started

Container	Image	Port
foodads-postgres-compose	postgres:latest	5432
foodads-ai-service-compose	Built from ./python-ai-service/Dockerfile	8000
foodads-backend-compose	Built from ./backend/Dockerfile	8080
foodads-frontend-compose	Built from ./frontend/Dockerfile	5173

Volumes

Volume	Purpose
postgres_data	PostgreSQL data persistence
hf_cache	HuggingFace model cache (avoids re-downloading on restart)

GPU Support

The AI service container is configured with `NVIDIA_VISIBLE_DEVICES=all` and the `nvidia` driver reservation, enabling GPU passthrough when the host has NVIDIA drivers and the Docker GPU toolkit installed.

Startup order: postgres → ai-service → backend → frontend

```
# Start the full stack
docker compose up --build

# Start in detached mode
docker compose up --build -d

# View logs
docker compose logs -f ai-service

# Stop everything
docker compose down
```

IMPORTANT: Before running, copy `.env.example` to `.env` and fill in your real `GROQ_API_KEY`, `HF_TOKEN`, and a strong `Jwt__Secret`.

4.2 Dockerfiles

Frontend Dockerfile

- Multi-stage: builds the Vite app with node, then serves the dist/ folder.

Backend Dockerfile

- Multi-stage: restores NuGet packages, builds with dotnet publish, runs with the ASP.NET runtime image.

Python AI Service Dockerfile

- Installs Python dependencies including PyTorch (CUDA 12.1 wheel), starts Uvicorn on port 8000.

4.3 Local Run Scripts

Script	Platform	Description
run-local.ps1	PowerShell	Starts backend, frontend, and python service locally without Docker
run.bat	Windows CMD	Batch file shortcut to run the stack

5. Environment Variables

All variables are set in the root .env file (copied from .env.example).

5.1 Shared / Docker Compose

Variable	Example	Description
APP_ENVIRONMENT	Development	Service environment mode
ASPNETCORE_ENVIRONMENT	Development	.NET environment

5.2 Backend (ASP.NET Core)

Variable	Example	Description
ConnectionStrings__FoodAdsAI	Host=postgres;Port=5432;...	PostgreSQL connection string
AiService__BaseUrl	http://ai-service:8000	Python AI service base URL
Jwt__Issuer	FoodAdsAI	JWT issuer claim
Jwt__Audience	FoodAdsAI	JWT audience claim
Jwt__Secret	replace-with-long-random-value	JWT signing secret (min 32 chars recommended)
Jwt__AccessTokenMinutes	60	Access token lifetime in minutes
Jwt__RefreshTokenDays	30	Refresh token lifetime in days

5.3 Python AI Service

Variable	Example	Description
GROQ_API_KEY	gsk_...	Groq API key for LLM prompt enhancement and campaign generation
BASE_MODEL_ID	stablediffusionapi/realistic-vision-v51	HuggingFace model ID for Stable Diffusion base
LORA_ADAPTER_PATH	/app	Path to directory containing LoRA adapter files
HF_TOKEN	hf_...	HuggingFace token for private model access
ALLOW_PLACEHOLDER_IMAGES	false	If true, serves placeholder images when model is not loaded
OUTPUT_DIR	/app/output	Directory where generated PNG files are saved

5.4 Frontend

Variable	Example	Description
VITE_API_URL	http://localhost:8080	Backend API base URL used by the React app

6. Frontend

Path: ..\FoodAds-Ai\frontend

Stack: React 19, TypeScript, Vite, React Router v6, React Hook Form, Zod, Axios, TanStack Query, Framer Motion, Lucide Icons, Recharts.

6.1 Pages & Routes

Route	Page	Auth	Description
/	LandingPage	No	Marketing landing page
/login	LoginPage	No	Email + password login
/register	RegisterPage	No	Account registration
/app	DashboardPage	Yes	Overview / stats dashboard
/app/generate	GeneratorPage	Yes	Main campaign generation studio
/app/history	HistoryPage	Yes	Saved campaign history with reports
/app/restaurants	RestaurantsPage	Yes	Manage restaurant brand profiles
*	NotFoundPage	No	404 fallback

6.2 Key Components

Component	Purpose
Layout.tsx	Shared sidebar + topbar shell for all authenticated pages
RequireAuth.tsx	Route guard — redirects to /login if not authenticated
StatCard.tsx	Reusable metric card used in Dashboard

6.3 Hooks & Utilities

File	Purpose
hooks/useAuth.tsx	Auth context — stores user session, login/logout helpers
hooks/useTheme.tsx	Dark/light theme context

File	Purpose
lib/api.ts	Axios API client with JWT bearer token injection and auto-refresh interceptor
lib/session.ts	Access/refresh token persistence in localStorage
lib/types.ts	All TypeScript interfaces (auth, campaigns, restaurants, history, etc.)

6.4 Generator Page (Core Feature)

The GeneratorPage is the main user-facing feature. It:

1. Lets the user type a prompt, select a restaurant brand profile (prefill), pick an image model (base or lora), and choose hardware (cuda / cpu).
2. On submit, calls three sequential APIs:
    - POST /api/generation/enhance-prompt → AI-enhanced prompt
    - POST /api/generation/image → generated food image (base64)
    - POST /api/generation/campaign → full marketing copy
3. Displays results in a 5-tab channel preview panel:
    - **Instagram Feed** — mock post with image, caption, hashtags, CTA
    - **Google Search** — mock ad with headline + description
    - **Email Newsletter** — mock email with subject, body, image embed, CTA button
    - **Mobile Preview** — SMS message + Push notification bubbles
    - **Content Calendar** — 3-day social content plan (Day 1 Teaser, Day 2 Hero, Day 3 Offer)
4. Shows a completion summary card with generation status for each output type.
5. Exposes buttons to:
    - Copy any individual section to clipboard
    - Copy Campaign Brief — full text summary to clipboard
    - Download Image — saves the PNG
    - Download Report — generates and downloads a full-styled HTML report with the image embedded

6.5 History Page

- Lists all saved campaigns from GET /api/generation/campaigns
- Each item shows: headline, model, prompt, date, and image preview
- Users can expand a campaign to see all copy fields
- Favorite/unfavorite campaigns via the API

6.6 Restaurants Page

- CRUD management of restaurant brand profiles
- Fields: Name, Cuisine Type, Brand Tone, Website URL, Logo URL

- Profiles are used to prefill the Generator form

6.7 Frontend Dependencies

```
"react": "^19.0.0"
"react-router-dom": "^6.26.1"
"axios": "^1.7.4"
"@tanstack/react-query": "^5.51.23"
"react-hook-form": "^7.52.2"
"zod": "^3.23.8"
"framer-motion": "^11.5.4"
"lucide-react": "^0.451.0"
"recharts": "^2.13.0"
```

---

7. Backend (ASP.NET Core)

Path: ..\FoodAds-Ai\backend

Stack: ASP.NET Core 8 Minimal API, Entity Framework Core 8, Npgsql, JWT Bearer Auth.

7.1 Solution Structure

```
backend/
  src/
    FoodAdsAI.Api/            ← Entry point, endpoints, middleware wiring
    FoodAdsAI.Application/    ← Services, DTOs, abstractions
    FoodAdsAI.Domain/         ← Entity models (plain C# classes)
    FoodAdsAI.Infrastructure/ ← EF Core DbContext, repositories, JWT, AI client
```

7.2 API Endpoints

All endpoints are under /api. Protected routes require a valid JWT Bearer token.

Auth — /api/auth

Method	Path	Auth	Description
POST	/api/auth/register	No	Register new user, returns tokens
POST	/api/auth/login	No	Login, returns access + refresh tokens
POST	/api/auth/refresh	No	Refresh access token using refresh token
POST	/api/auth/logout	No	Invalidates the refresh token
GET	/api/auth/me	Yes	Returns current user info

Generation — /api/generation

Method	Path	Auth	Description
POST	/api/generation/image	Yes	Generate food image (proxies to Python AI)
POST	/api/generation/campaign	Yes	Generate marketing campaign copy
POST	/api/generation/enhance-prompt	Yes	Enhance a raw user prompt
POST	/api/generation/caption	Yes	Alias for enhance-prompt
POST	/api/generation/video-script	Yes	Stub endpoint for reel scripts
GET	/api/generation/campaigns	Yes	List saved campaigns

History — /api/history

Method	Path	Auth	Description
GET	/api/history	Yes	List prompt history for the current user
DELETE	/api/history/{id}	Yes	Delete a history entry

Favorites — /api/favorites

Method	Path	Auth	Description
GET	/api/favorites	Yes	List favorites for the current user
POST	/api/favorites	Yes	Add a favorite (by campaignId or imageId)
POST	/api/favorites/campaign/{id}	Yes	Favorite a specific campaign
DELETE	/api/favorites/{id}	Yes	Remove a favorite

Restaurants — /api/restaurants

Method	Path	Auth	Description
GET	/api/restaurants	Yes	List all restaurants
POST	/api/restaurants	Yes	Create a new restaurant profile
PUT	/api/restaurants/{id}	Yes	Update a restaurant profile

7.3 Application Services

Service	Responsibility
AuthService	Register, login, refresh, logout, password hashing (ASP.NET Identity PasswordHasher), role assignment

Service	Responsibility
GenerationService	Proxies image + campaign requests to Python AI service via PythonAiClient, persists results
RestaurantService	CRUD for restaurant profiles

7.4 Infrastructure

Component	Purpose
FoodAdsAiDbContext	EF Core DbContext — all tables, fluent configuration, soft-delete, audit timestamps
EfRepositoryStore	Generic EF-backed repository implementing IRepositoryStore
EfAuthRepository	Auth-specific queries (find by email, by refresh token hash)
JwtTokenService	Issues signed JWT access tokens + opaque refresh tokens
PythonAiClient	Typed HTTP client that proxies calls to the Python AI service with snake_case serialization
FoodAdsAiDbSeeder	Seeds roles (Admin, RestaurantOwner) and demo users/restaurants (disabled by default in Program.cs)

7.5 Authentication Flow

6. User registers or logs in → backend creates a JWT access token (configurable expiry, default 60 min) and an opaque refresh token (default 30 days).
7. The refresh token is hashed and stored in the users table.
8. Frontend auto-refreshes the access token on 401 responses using the stored refresh token.
9. Logout clears the refresh token hash from the database.

8. Python AI Service

Path: `d:\FoodAds-Ai\python-ai-service`

Stack: FastAPI, Uvicorn, PyTorch, Diffusers (Stable Diffusion), PEFT (LoRA), Groq SDK, Pillow.

8.1 API Endpoints (prefix: /v1)

Method	Path	Description
GET	/v1/health	Service health: device, model loaded status, load error

Method	Path	Description
GET	/v1/ready	Ready check — true if model is loaded or placeholder allowed
POST	/v1/prompt/enhance	Enhance a raw prompt via Groq LLM
POST	/v1/images/generate	Generate food images
POST	/v1/campaigns/generate	Generate marketing campaign copy

8.2 Services

PromptEnhancer

- **Primary:** Sends the raw user prompt to Groq (llama-3.1-8b-instant) with a system prompt instructing it to add lighting, composition, camera, and set-dressing details.
- **Fallback (no Groq key):** Builds a structured prompt string from template with RAW photo, style keywords, 8k uhd, DSLR, sharp focus, Fujifilm XT3.
- Always pairs the enhanced prompt with a hardcoded negative prompt (filters: oversaturated, cartoon, grain, watermark, etc.).

MarketingService

- **Primary:** Calls Groq (llama-3.1-8b-instant) requesting JSON output with all 16 campaign fields.
- **Output fields:** headline, caption, cta, hashtags, google_ads_copy, facebook_post, instagram_post, tiktok_caption, reel_script, email_subject, email_body, sms_message, push_notification, menu_description, seo_description, promotional_offer, content_calendar.
- **Fallback (no Groq key / error):** Returns rule-based template copy using restaurant name and cuisine.

GenerationService

- Orchestrates prompt enhancement → image generation pipeline.
- Supports three image model modes:
  - base — Stable Diffusion base pipeline
  - lora — LoRA fine-tuned pipeline (falls back to base if adapter missing)
  - custom — either HuggingFace Hub model (loaded on demand) or OpenAI DALL-E API
- Uses a threading.Lock to serialize GPU access.
- Saves generated PNGs to the configured OUTPUT_DIR.
- Returns images as base64-encoded strings in the response.

ModelRegistry

- Loaded once at app startup via FastAPI lifespan.
- Validates CUDA with a real tensor operation (not just is_available()) to avoid false positives in containers.

- Loads Stable Diffusion base pipeline (StableDiffusionPipeline.from_pretrained) with DPMSolverMultistepScheduler (Karras sigmas, dpmsolver++).
- Loads LoRA adapter using PEFT's PeftModel.from_pretrained on top of the base UNet.
- Builds the Groq client if GROQ_API_KEY is set.
- If model loading fails and ALLOW_PLACEHOLDER_IMAGES=true, generates a placeholder image (styled box with prompt text) instead of erroring.

8.3 Python Dependencies

```
fastapi >= 0.115
uvicorn[standard] >= 0.30
pydantic >= 2.8
pydantic-settings >= 2.4
httpx >= 0.27
Pillow >= 10.4
groq >= 0.9
torch >= 2.3  (CUDA 12.1 wheel)
torchvision >= 0.18
transformers >= 4.43
diffusers >= 0.30
accelerate >= 0.33
peft >= 0.12
```

8.4 LoRA Adapter

The repository includes a pre-trained LoRA adapter for food photography style:

- adapter_config.json — adapter config (rank, target modules, etc.)
- adapter_model.safetensors — adapter weights (~13 MB)

Base model: `stablediffusionapi/realistic-vision-v51` (configured via `BASE_MODEL_ID` env var).

9. Database

Engine: PostgreSQL (latest)

ORM: Entity Framework Core 8 with Npgsql provider

Migrations: Code-first, migration file at backend/src/FoodAdsAI.Infrastructure/Persistence/Migrations/.

9.1 Tables

Table	Purpose
users	User accounts — email, display name, password hash, refresh token hash
roles	Roles (Admin, RestaurantOwner)

Table	Purpose
user_roles	Many-to-many join between users and roles
restaurants	Restaurant brand profiles (name, cuisine, tone, URLs)
campaigns	Saved generated campaigns with all copy fields + embedded image
generated_images	Individual generated image records (base64, filename, model)
prompt_history	Log of all prompt enhancement calls per user
favorites	User-favorited campaigns or images
subscriptions	Subscription plan per user (plan name, status)
api_usage	Per-user operation log with estimated cost tracking
notifications	In-app notification records per user

9.2 Cross-Cutting Behaviors

- **Soft delete** on all user-facing entities (IsDeleted, DeletedAt) — deletes set IsDeleted = true instead of removing rows.
- **Audit timestamps** auto-applied on every save: CreatedAt, UpdatedAt.
- **Global query filters** exclude soft-deleted records from all queries by default.
- **Auto-migration** on backend startup (MigrateAsync if migrations exist, else EnsureCreatedAsync).

9.3 Entity Relationships

```
UserAccount ──< UserRole >── Role
UserAccount ──< Subscription
UserAccount ──< ApiUsage
UserAccount ──< Notification
UserAccount ──< Favorite ──> Campaign
UserAccount ──< Favorite ──> GeneratedImage
Restaurant  ──< Campaign ──< GeneratedImage
Restaurant  ──< PromptHistory
UserAccount ──< PromptHistory
```

10. Appendix A — Data Flow for a Single Generation

```
User clicks "Generate" in browser
  │
  ▼
POST /api/generation/enhance-prompt (Backend)
  │  → PythonAiClient → POST /v1/prompt/enhance (Python)
  │       → PromptEnhancer → Groq LLaMA 3.1
  │  ← returns { enhancedPrompt, negativePrompt, source }
  │
```

```
  ▼
POST /api/generation/image (Backend)
  │  → PythonAiClient → POST /v1/images/generate (Python)
  │       → GenerationService → StableDiffusion pipeline (GPU)
  │       → saves PNG to OUTPUT_DIR
  │  ← returns { generationId, model, images[{ fileName, base64Data }] }
  │
  ▼
POST /api/generation/campaign (Backend)
  │  → PythonAiClient → POST /v1/campaigns/generate (Python)
  │       → MarketingService → Groq LLaMA 3.1 (JSON mode)
  │  ← returns { headline, caption, cta, hashtags, googleAdsCopy,
  │              facebookPost, instagramPost, tiktokCaption, reelScript,
  │              emailSubject, emailBody, smsMessage, pushNotification,
  │              menuDescription, seoDescription, promotionalOffer,
  │              contentCalendar }
  │
  ▼
Frontend renders 5-tab channel preview + completion summary card
User can copy, download image, or download full HTML report
```

11. AI Model Research — Food Image Generation (SD + LoRA)

Notebook: `Food Image Generation-SD+LoRA.ipynb`

Platform: Kaggle (GPU: NVIDIA Tesla T4, 15.6 GB VRAM)

Runtime: Python 3.12, PyTorch 2.x, CUDA

This section documents the full research pipeline used to develop and evaluate the LoRA-fine-tuned Stable Diffusion model that powers FoodAds AI's image generation feature.

11.1 Data Collection & Preprocessing

Source Dataset

- **Dataset:** `jxie/coco_captions` — streamed from HuggingFace Hub.
- **Scan scope:** Up to 600,000 COCO samples were scanned in streaming mode.
- **Food filtering:** A keyword-based filter (FOOD_KEYWORDS) matched captions containing terms like pizza, burger, cake, sushi, salad, soup, bread, coffee, etc.
- **Collected samples:** 16,782 unique food images (out of 566,747 scanned).
- Images were resized to max 512×512 and saved as JPEG (quality=85).

CLIP-Based Relevance Scoring

To ensure images were genuinely food-focused (not people eating, restaurant interiors, etc.), each image was scored using OpenAI CLIP (openai/clip-vit-base-patch32):

- **Food probes:** 5 text descriptions of centered food photos.

- **Non-food probes:** 7 descriptions of people, scenes, interiors.
- **Threshold:** CLIP score ≥ 0.50 → image is food-relevant.
- **Results:** 11,108 out of 16,782 images passed (66%).

Category Mapping & Balancing

Images were mapped to 24 food categories (pizza, cake, sandwich, donut, banana, hot dog, broccoli, apple, bread, coffee, rice, carrot, salad, soup, chicken, pasta, fish, burger, etc.).

- Dropped categories with fewer than 50 samples.
- Capped at 1,500 samples per category to prevent class imbalance.
- **Final balanced dataset:** 10,036 images across 19 categories.

Train / Val / Test Split

Split	Samples
Train	9,032
Validation	502
Test	502

Data Augmentation (Training Transforms)

```
T.Resize(512, interpolation=BICUBIC)
T.CenterCrop(512)
T.RandomHorizontalFlip(p=0.5)
T.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1)
T.RandomRotation(degrees=5)
T.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
```

Caption enrichment: training captions were prefixed with "a RAW food photo of" and suffixed with ", DSLR, sharp focus, high quality, 8k" to align with the photorealism style keywords used at inference.

11.2 Model Architecture & Techniques

Base Model

- `stablediffusionapi/realistic-vision-v51` — a Stable Diffusion 1.5-based model fine-tuned for photorealistic outputs.
- Components: CLIP text encoder, VAE (encoder/decoder), UNet (denoising backbone).

LoRA (Low-Rank Adaptation)

LoRA was applied exclusively to the UNet to adapt the denoising backbone to food photography without retraining the full model.

LoRA Configuration

Parameter	Value
Rank (r)	8
Alpha (lora_alpha)	16
Dropout	0.05
Target modules	to_q, to_k, to_v, to_out.0, proj_in, proj_out, ff.net.0.proj, ff.net.2

- **Trainable parameters:** 3,391,488 (≈3.39M out of 862.9M total = 0.39%).
- VAE and CLIP text encoder were frozen (requires_grad_(False)).
- unet.enable_gradient_checkpointing() and vae.enable_slicing() were used to reduce VRAM usage.

Attention Mechanism

LoRA targets the cross-attention and self-attention layers of the UNet (to_q, to_k, to_v, to_out.0) and the feed-forward projection layers (proj_in, proj_out, ff.net.0.proj, ff.net.2). These are the primary layers where text-image alignment is learned.

Noise Scheduler

- **Training:** DDPMScheduler (Denoising Diffusion Probabilistic Model) — adds Gaussian noise at random timesteps during training.
- **Inference:** DPMSolverMultistepScheduler with use_karras_sigmas=True and algorithm_type="dpmsolver++" — a fast, high-quality sampler (40 steps achieves results comparable to 100+ DDPM steps).

LLM Prompt Enhancement (Groq + LLaMA 3.1)

At inference time, raw user requests are enhanced via Groq API (llama-3.1-8b-instant) before being passed to Stable Diffusion:

- System prompt instructs the LLM to act as an expert Stable Diffusion prompt engineer for food photography.
- Always starts with "RAW photo," and ends with "food photography, 8k uhd, DSLR, sharp focus, high quality, Fujifilm XT3".
- Adds: lighting, composition angle, surface/background, and specific food details.

Example Prompt Enhancement

User Request	Enhanced Prompt
a slice of pepperoni pizza on a white plate	RAW photo, close-up shot of a warm golden-brown slice of pepperoni pizza, freshly baked with melted mozzarella cheese and gooey pepperoni rings, on a crisp white ceramic plate, placed on a wooden background with a soft warm glow of natural light...
a chocolate birthday cake with candles	RAW photo, a deliciously moist and rich chocolate birthday cake with colorful candles, sitting atop a vintage white ceramic plate on a

User Request	Enhanced Prompt
	wooden board in a warm and inviting restaurant lighting, captured from a 45-degree angle...

Negative Prompt (Hardcoded)

```
oversaturated, ugly, 3d, render, cartoon, grain, low-res, kitsch, blurred,
soft, deformed, extra limbs, close up, b&w, weird colors, watermark, blur,
text, writing, logo, oversharpen
```

11.3 Model Training

Hyperparameters

Parameter	Value
Epochs	3
Batch size	16
Gradient accumulation steps	4 (effective batch = 64)
Learning rate	1e-4
LR schedule	Cosine decay with linear warmup
Warmup steps	150
Gradient clipping	1.0
Optimizer	AdamW (weight_decay=1e-2)
Mixed precision	fp16 (autocast + GradScaler)
Loss function	MSE between predicted and actual noise

Training Loop

Each training step:

10. Encode image with frozen VAE → latents (scaled by 0.18215).
11. Sample random Gaussian noise.
12. Sample random timestep t from [0, 1000).
13. Add noise to latents via DDPMScheduler.add_noise.
14. Encode caption with frozen CLIP text encoder.
15. UNet (LoRA) predicts the noise.
16. Compute MSE loss between predicted and actual noise.
17. Backprop through LoRA parameters only.

Training Results

Epoch	Train Loss	Val Loss	LR	Time
1	0.1678	0.1620	9.47e-05	67.1 min (Best saved)
2	0.1666	0.1621	5.14e-05	67.4 min
3	0.1660	0.1620	2.98e-08	67.6 min (Best saved)

- **Best epoch:** 3 | **Best validation loss:** 0.1620
- **Total training time:** ~202 minutes on Tesla T4

Saved Artifacts

- Best LoRA adapter checkpoint saved to: /kaggle/working/sd_lora/lora_best/
- Adapter config and weights exported as adapter_config.json + adapter_model.safetensors (~13 MB).

11.4 Quantitative Evaluation: FID & Inception Score

Three model variants were compared against 502 real food images from the test set:

Model	IS Mean ↑	IS Std	FID ↓
Baseline SD (no LoRA)	4.606	±0.490	175.087
SD + LoRA	5.117	±0.648	171.946
SD + LoRA + Groq Prompt Enhancement	5.117	±0.648	171.946

Evaluation Protocol

- 8 food prompts × 15 seeds = 120 generated images per model.
- Metrics computed using torch-fidelity library.
- **IS (Inception Score) ↑** higher is better — measures image diversity and quality.
- **FID (Fréchet Inception Distance) ↓** lower is better — measures distributional similarity to real images.

Key Findings

- LoRA fine-tuning improved IS by +0.511 and reduced FID by -3.141 compared to the baseline.
- Prompt enhancement via Groq/LLaMA 3.1 produced visually richer images (more accurate composition, lighting, and detail) but did not change FID/IS since the enhanced prompts generated the same underlying images as LoRA for these test seeds.

11.5 Challenges Encountered

Challenge	Description	Resolution
VRAM constraints	Tesla T4 has only 15.6 GB; loading full SD model (VAE + CLIP + UNet) consumed ~2.18 GB before training.	Used fp16, gradient checkpointing, VAE slicing, and gradient accumulation (effective batch=64 with bs=16 + accum=4).
Dataset noise	COCO captions are not food-specific — many images showed people eating, restaurant scenes, not the food itself.	Implemented CLIP-based relevance scoring to filter images; only kept samples with food CLIP score ≥ 0.50.
Class imbalance	Food categories were highly unbalanced (pizza had thousands of samples, burger had <100).	Applied per-category cap of 1,500 samples and stratified train/val/test split.
Streaming dataset size	Full COCO dataset is massive; scanning all of it would take hours.	Capped scan at 600,000 samples and used HuggingFace streaming to avoid downloading the full dataset.
Slow training	Each epoch took ~67 minutes due to image encoding + UNet forward pass on every batch.	Frozen VAE/CLIP encoders, fp16 mixed precision, and gradient checkpointing reduced VRAM and allowed larger effective batches.
FID plateau	FID did not improve significantly beyond epoch 1.	Trained for 3 epochs with cosine LR decay; best checkpoint at epoch 3 showed marginal improvement (val_loss 0.1620 → 0.1620).

11.6 Libraries Used in Notebook

```
diffusers >= 0.30       — Stable Diffusion pipeline, schedulers (DDPMScheduler,
DPMSolverMultistepScheduler), UNet, VAE
transformers >= 4.43    — CLIPTokenizer, CLIPTextModel, CLIPProcessor, CLIPModel
accelerate >= 0.33      — Mixed precision training utilities
peft >= 0.12            — LoraConfig, get_peft_model, PeftModel (LoRA fine-tuning)
torch >= 2.3            — Core deep learning framework (CUDA 12.1)
torchvision >= 0.18     — Image transforms and augmentations
torch-fidelity          — FID and Inception Score computation
groq >= 0.9             — Groq API client for LLaMA 3.1 prompt enhancement
datasets                — HuggingFace streaming dataset loader
Pillow >= 10.4          — Image I/O and processing
wordcloud               — Caption word cloud visualization
scikit-learn            — train_test_split for stratified splitting
pandas, numpy           — Data manipulation and statistics
matplotlib              — Training plots, EDA charts, result grids
tqdm                    — Progress bars
```

— End of Document —